Delta Lake MERGE Assignment

This notebook shows how to load CSV data, clean it, write it as a Delta table, apply a MERGE operation, and validate the final result.

Step 1: Import required libraries

```
from pathlib import Path
from pyspark.sql import SparkSession

project_root = Path(r"D:\Samradni Dahiphale\Projects\Celebal task\
Data_Engineering_Internship_Celebal_Technologies\Assignment 7\delta-
lake-assignment")

delta_jar = project_root / "delta-spark_2.12-3.2.0.jar"
delta_storage_jar = project_root / "delta-storage-3.2.0.jar"

assert delta_jar.exists(), delta_jar
assert delta_storage_jar.exists(), delta_storage_jar

spark = (
    SparkSession.builder
    .appName("delta-scd-assignment")
    .master("local[*]")
    .config("spark.jars", f"{delta_jar},{delta_storage_jar}")
    .config("spark.sql.extensions",
"io.delta.sql.DeltaSparkSessionExtension")
    .config(
        "spark.sql.catalog.spark_catalog",
        "org.apache.spark.sql.delta.catalog.DeltaCatalog"
    )
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")

from pathlib import Path
from pyspark.sql import SparkSession
from delta import *
```

Step 2: Create Spark session

```
if 'spark' in locals():
    spark.stop()

project_root = Path.cwd().resolve()
delta_jar = project_root / "delta-spark_2.12-3.2.0.jar"
delta_storage_jar = project_root / "delta-storage-3.2.0.jar"
```

```
spark = (
    SparkSession.builder
    .appName("delta-scd-assignment")
    .master("local[*]")
    .config("spark.jars", f"{delta_jar},{delta_storage_jar}")
    .config("spark.sql.extensions",
"io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")
spark
```

```
<pyspark.sql.session.SparkSession at 0x2373e614b90>
```

Step 3: Set project paths

```
current_dir = Path.cwd().resolve()

if (current_dir / "data").exists():
    project_root = current_dir
elif (current_dir.parent / "data").exists():
    project_root = current_dir.parent
else:
    project_root = current_dir

data_dir = project_root / "data"
master_csv = data_dir / "Sample - Superstore.csv"
incremental_csv = data_dir / "superstore_incremental.csv"
delta_table_path = project_root / "delta_tables" / "superstore_master"

project_root, data_dir, master_csv, incremental_csv, delta_table_path
```

```
(WindowsPath('D:/Samradni Dahiphale/Projects/Celebal
task/Data_Engineering_Internship_Celebal_Technologies/Assignment
7/delta-lake-assignment'),
 WindowsPath('D:/Samradni Dahiphale/Projects/Celebal
task/Data_Engineering_Internship_Celebal_Technologies/Assignment
7/delta-lake-assignment/data'),
 WindowsPath('D:/Samradni Dahiphale/Projects/Celebal
task/Data_Engineering_Internship_Celebal_Technologies/Assignment
7/delta-lake-assignment/data/Sample - Superstore.csv'),
 WindowsPath('D:/Samradni Dahiphale/Projects/Celebal
task/Data_Engineering_Internship_Celebal_Technologies/Assignment
7/delta-lake-assignment/data/superstore_incremental.csv'),
 WindowsPath('D:/Samradni Dahiphale/Projects/Celebal
```

```
task/Data_Engineering_Internship_Celebal_Technologies/Assignment
7/delta-lake-assignment/delta_tables/superstore_master'))
```

Step 4: Read the master CSV

Please take a screenshot of the displayed dataset and save it in the screenshots/data_loading folder.

```
from pathlib import Path
from pyspark.sql import SparkSession

project_root = Path.cwd()
delta_jar = project_root / "delta-spark_2.12-3.2.0.jar"
delta_storage_jar = project_root / "delta-storage-3.2.0.jar"

spark = (
    SparkSession.builder
    .appName("delta-scd-assignment")
    .master("local[*]")
    .config("spark.jars", f"{delta_jar},{delta_storage_jar}")
    .config(
        "spark.sql.extensions",
        "io.delta.sql.DeltaSparkSessionExtension"
    )
    .config(
        "spark.sql.catalog.spark_catalog",
        "org.apache.spark.sql.delta.catalog.DeltaCatalog"
    )
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")

master_df = spark.read.option("header", True).option("inferSchema",
True).csv(str(master_csv))
master_df.show(10)
```

```
+------+--------------+----------+----------+--------------
+-----------+---------------+---------+-------------+---------------
+----------+-----------+------+---------------+--------------
+------------+--------------------+--------+--------+--------+--------
+
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer
ID|  Customer Name|  Segment|      Country|           City|     State|
Postal Code|Region|     Product ID|       Category|Sub-Category|
Product Name|   Sales|Quantity|Discount|  Profit|
+------+--------------+----------+----------+--------------
+-----------+---------------+---------+-------------+---------------
+----------+-----------+------+---------------+--------------
```

```
+------------+-------------------+--------+--------+--------+--------
+
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-
12520|    Claire Gute| Consumer|United States|      Henderson|
Kentucky|      42420| South|FUR-BO-10001798|      Furniture|
Bookcases|Bush Somerset Col...|  261.96|       2|       0| 41.9136|
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-
12520|    Claire Gute| Consumer|United States|      Henderson|
Kentucky|      42420| South|FUR-CH-10000454|      Furniture|
Chairs|Hon Deluxe Fabric...|  731.94|       3|       0| 219.582|
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-
13045|Darrin Van Huff|Corporate|United States|    Los Angeles|
California|      90036|  West|OFF-LA-10000240|Office Supplies|
Labels|Self-Adhesive Add...|   14.62|       2|       0|  6.8714|
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-
20335| Sean O'Donnell| Consumer|United States|Fort Lauderdale|
Florida|      33311| South|FUR-TA-10000577|      Furniture|
Tables|Bretford CR4500 S...|957.5775|       5|    0.45|-383.031|
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-
20335| Sean O'Donnell| Consumer|United States|Fort Lauderdale|
Florida|      33311| South|OFF-ST-10000760|Office Supplies|
Storage|Eldon Fold 'N Rol...|  22.368|       2|     0.2|  2.5164|
|     6|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-
11710|Brosina Hoffman| Consumer|United States|    Los Angeles|
California|      90032|  West|FUR-FU-10001487|      Furniture|
Furnishings|Eldon Expressions...|   48.86|       7|       0| 14.1694|
|     7|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-
11710|Brosina Hoffman| Consumer|United States|    Los Angeles|
California|      90032|  West|OFF-AR-10002833|Office Supplies|
Art|          Newell 322|    7.28|       4|       0|  1.9656|
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-
11710|Brosina Hoffman| Consumer|United States|    Los Angeles|
California|      90032|  West|TEC-PH-10002275|     Technology|
Phones|Mitel 5320 IP Pho...| 907.152|       6|     0.2| 90.7152|
|     9|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-
11710|Brosina Hoffman| Consumer|United States|    Los Angeles|
California|      90032|  West|OFF-BI-10003910|Office Supplies|
Binders|DXL Angle-View Bi...|  18.504|       3|     0.2|  5.7825|
|    10|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-
11710|Brosina Hoffman| Consumer|United States|    Los Angeles|
California|      90032|  West|OFF-AP-10002892|Office Supplies|
Appliances|Belkin F5C206VTEL...|   114.9|       5|       0|   34.47|
+------+-------------+----------+----------+-------------
+-----------+--------------+---------+-------------+--------------
+----------+-----------+------+--------------+--------------
+------------+-------------------+--------+--------+--------+--------
+
only showing top 10 rows
```

Step 5: Write the data as a Delta table

Please take a screenshot of the successful Delta table creation and save it in the screenshots/data_loading folder.

```
import re

master_df = master_df.toDF(*[
    re.sub(r"[^A-Za-z0-9_]", "_", column).strip("_")
    for column in master_df.columns
])

master_df.printSchema()
```

```
root
 |-- Row_ID: integer (nullable = true)
 |-- Order_ID: string (nullable = true)
 |-- Order_Date: string (nullable = true)
 |-- Ship_Date: string (nullable = true)
 |-- Ship_Mode: string (nullable = true)
 |-- Customer_ID: string (nullable = true)
 |-- Customer_Name: string (nullable = true)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- State: string (nullable = true)
 |-- Postal_Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product_ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub_Category: string (nullable = true)
 |-- Product_Name: string (nullable = true)
 |-- Sales: string (nullable = true)
 |-- Quantity: string (nullable = true)
 |-- Discount: string (nullable = true)
 |-- Profit: double (nullable = true)
```

```
master_df.write.format("delta").mode("overwrite").save(str(delta_table
_path))
print("Delta table created successfully at:", delta_table_path)
```

```
Delta table created successfully at: D:\Samradni Dahiphale\Projects\
Celebal task\Data_Engineering_Internship_Celebal_Technologies\
Assignment 7\delta-lake-assignment\delta_tables\superstore_master
```

Step 6: Clean the data

Please take a screenshot of the cleaned table and save it in the screenshots/data_cleaning folder.

```
cleaned_df = (
    master_df
    .withColumnRenamed("Row_ID", "ID")
    .withColumnRenamed("Order_ID", "OrderID")
    .withColumnRenamed("Customer_ID", "CustomerID")
    .withColumnRenamed("Customer_Name", "CustomerName")
    .withColumnRenamed("Postal_Code", "PostalCode")
    .dropDuplicates(["ID", "OrderID", "CustomerID"])
    .na.fill({"City": "Unknown", "State": "Unknown"})
)

cleaned_df.select(
    "ID", "OrderID", "CustomerID", "CustomerName", "City", "State",
"Sales"
).show(10, truncate=False)

(
    cleaned_df.write
    .format("delta")
    .mode("overwrite")
    .option("overwriteSchema", "true")
    .save(str(delta_table_path))
)

print("Cleaned data written to Delta table at:", delta_table_path)
```

```
+---+--------------+----------+---------------+---------------
+----------+--------+
|ID |OrderID       |CustomerID|CustomerName   |City           |State
|Sales   |
+---+--------------+----------+---------------+---------------
+----------+--------+
|1  |CA-2016-152156|CG-12520  |Claire Gute    |Henderson      |
Kentucky  |261.96  |
|2  |CA-2016-152156|CG-12520  |Claire Gute    |Henderson      |
Kentucky  |731.94  |
|3  |CA-2016-138688|DV-13045  |Darrin Van Huff|Los Angeles    |
California|14.62   |
|4  |US-2015-108966|SO-20335  |Sean O'Donnell |Fort Lauderdale|Florida
|957.5775|
|5  |US-2015-108966|SO-20335  |Sean O'Donnell |Fort Lauderdale|Florida
|22.368  |
|6  |CA-2014-115812|BH-11710  |Brosina Hoffman|Los Angeles    |
California|48.86   |
|7  |CA-2014-115812|BH-11710  |Brosina Hoffman|Los Angeles    |
California|7.28    |
|8  |CA-2014-115812|BH-11710  |Brosina Hoffman|Los Angeles    |
California|907.152 |
|9  |CA-2014-115812|BH-11710  |Brosina Hoffman|Los Angeles    |
California|18.504  |
```

```
|10 |CA-2014-115812|BH-11710  |Brosina Hoffman|Los Angeles    |
California|114.9   |
+---+--------------+----------+---------------+--------------
+----------+--------+
only showing top 10 rows

Cleaned data written to Delta table at: D:\Samradni Dahiphale\
Projects\Celebal task\
Data_Engineering_Internship_Celebal_Technologies\Assignment 7\delta-
lake-assignment\delta_tables\superstore_master
```

Step 7: Read the incremental CSV

Please take a screenshot of the incremental data and save it in the screenshots/scd1 folder.

```
incremental_df = spark.read.option("header",
True).option("inferSchema", True).csv(str(incremental_csv))
incremental_df.show(10)

+-----+--------------+----------+------------+--------+-------------
+--------+--------+-----+--------+--------+------+
|   ID|       OrderID|CustomerID|CustomerName| Segment|      Country|
City|   State|Sales|Quantity|Discount|Profit|
+-----+--------------+----------+------------+--------+-------------
+--------+--------+-----+--------+--------+------+
|    1|CA-2016-152156|  CG-12520| Claire Gute|Consumer|United States|
New York|New York|320.0|       2|     0.0|  55.0|
|99999| NEW-2026-0001|  ZZ-99999|New Customer|Consumer|United States|
Chicago|Illinois|180.0|       1|     0.0|  30.0|
+-----+--------------+----------+------------+--------+-------------
+--------+--------+-----+--------+--------+------+
```

Step 8: Apply MERGE

Please take a screenshot of the MERGE execution result and save it in the screenshots/scd1 folder.

```
from delta.tables import DeltaTable

incremental_cleaned_df = (
    incremental_df
    .withColumnRenamed("ID", "ID")
    .withColumnRenamed("OrderID", "OrderID")
    .withColumnRenamed("CustomerID", "CustomerID")
    .withColumnRenamed("CustomerName", "CustomerName")
    .withColumnRenamed("Country", "Country")
    .withColumnRenamed("City", "City")
```

```
    .withColumnRenamed("State", "State")
    .withColumnRenamed("Sales", "Sales")
    .withColumnRenamed("Quantity", "Quantity")
    .withColumnRenamed("Discount", "Discount")
    .withColumnRenamed("Profit", "Profit")
)

delta_table = DeltaTable.forPath(spark, str(delta_table_path))
(
    delta_table.alias("target")
    .merge(incremental_cleaned_df.alias("source"), "target.ID =
source.ID")
    .whenMatchedUpdate(set={
        "City": "source.City",
        "State": "source.State",
        "Sales": "source.Sales",
        "Profit": "source.Profit",
    })
    .whenNotMatchedInsert(values={
        "ID": "source.ID",
        "OrderID": "source.OrderID",
        "CustomerID": "source.CustomerID",
        "CustomerName": "source.CustomerName",
        "Segment": "source.Segment",
        "Country": "source.Country",
        "City": "source.City",
        "State": "source.State",
        "Sales": "source.Sales",
        "Quantity": "source.Quantity",
        "Discount": "source.Discount",
        "Profit": "source.Profit",
    })
    .execute()
)

print("MERGE completed successfully")
```

```
MERGE completed successfully
```

Step 9: Show the final Delta table

Please take a screenshot of the final output and save it in the screenshots/final_output folder.

```
final_df = spark.read.format("delta").load(str(delta_table_path))
final_df.orderBy("ID").select("ID", "CustomerName", "City", "State",
"Sales", "Profit").show(truncate=False)
```

```
+---+------------------+--------------+-------------+--------
+--------+
```

```
|ID |CustomerName      |City            |State          |Sales   |Profit
|
+---+------------------+----------------+---------------+--------
+--------+
|1  |Claire Gute       |New York        |New York       |320.0   |55.0
|
|2  |Claire Gute       |Henderson       |Kentucky       |731.94  |
219.582 |
|3  |Darrin Van Huff   |Los Angeles     |California     |14.62   |6.8714
|
|4  |Sean O'Donnell    |Fort Lauderdale|Florida        |957.5775|-
383.031|
|5  |Sean O'Donnell    |Fort Lauderdale|Florida        |22.368  |2.5164
|
|6  |Brosina Hoffman   |Los Angeles     |California     |48.86   |
14.1694 |
|7  |Brosina Hoffman   |Los Angeles     |California     |7.28    |1.9656
|
|8  |Brosina Hoffman   |Los Angeles     |California     |907.152 |
90.7152 |
|9  |Brosina Hoffman   |Los Angeles     |California     |18.504  |5.7825
|
|10 |Brosina Hoffman   |Los Angeles     |California     |114.9   |34.47
|
|11 |Brosina Hoffman   |Los Angeles     |California     |1706.184|
85.3092 |
|12 |Brosina Hoffman   |Los Angeles     |California     |911.424 |
68.3568 |
|13 |Andrew Allen      |Concord         |North Carolina|15.552  |5.4432
|
|14 |Irene Maddox      |Seattle         |Washington     |407.976 |
132.5922|
|15 |Harold Pawlan     |Fort Worth      |Texas          |68.81   |-
123.858|
|16 |Harold Pawlan     |Fort Worth      |Texas          |2.544   |-3.816
|
|17 |Pete Kriz         |Madison         |Wisconsin      |665.88  |
13.3176 |
|18 |Alejandro Grove   |West Jordan     |Utah           |55.5    |9.99
|
|19 |Zuschuss Donatelli|San Francisco  |California     |8.56    |2.4824
|
|20 |Zuschuss Donatelli|San Francisco  |California     |213.48  |16.011
|
+---+------------------+----------------+---------------+--------
+--------+
only showing top 20 rows
```

Step 10: Validate the result

Please take a screenshot of the validation output and save it in the screenshots/validation folder.

```
row_count = final_df.count()
duplicate_ids = final_df.groupBy("ID").count().filter("count >
1").count()

print("Row count:", row_count)
print("Duplicate IDs:", duplicate_ids)
```

```
Row count: 9995
Duplicate IDs: 0
```

End of notebook

You can now stop the Spark session if you want to free resources.

```
spark.stop()
```